SAVEETHA SCHOOL OF ENGINEERING

SIMATS Engineering

Department of Computer Science and Engineering

OPERATING SYSTEMS

ASSIGNMENT 2

CO2: Analyse process management and deadlock-handling techniques to improve CPU utilisation and system performance – processes, process scheduling, threads, inter-process communication, synchronisation using semaphores, monitors and the critical-section problem, and deadlock handling including the Banker's algorithm.

(Total Marks – 50)

Name	Haakesh C
Register Number	1972260066
Course Code	CSA0403
Course Name	Operating Systems
Assignment	Assignment 2 – CO2

Q1 – Processes and Process States

Q1 (K5 – Evaluate) **[6 Marks]**

Context: Online examination server – one process per student, plus auto-save and proctoring processes. With 2,000 students the server becomes slow and sessions hang.

Challenge: Draw the process-state diagram and the PCB; write a fork() program and observe it with ps and /proc.

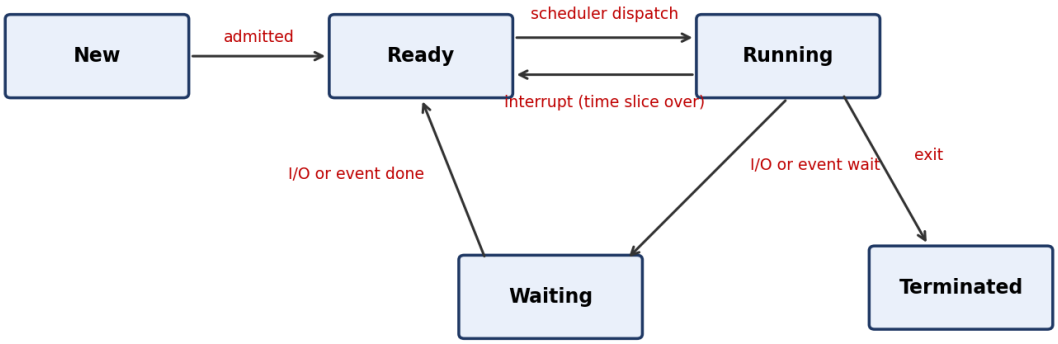
Evaluation: where does the time go with 2,000 active processes (context switch, memory, I/O wait)? Is one process per student sound?

Design Decision: one process per student, one thread per student or a fixed pool of worker processes?

Alternative Concept: each session as a thread inside one server process – evaluate context-switch time, memory per session and the effect of a crash.

Answer:

1.1 Process-state diagram



New: the process is being created. **Ready:** it has everything it needs and is waiting for the CPU. **Running:** its instructions are being executed. **Waiting:** it is blocked for an I/O or an event (for example a student's answer being saved to disk). **Terminated:** it has finished and the OS is cleaning up. A student session mostly moves between Running and Waiting, because it waits for the student and for the disk.

1.2 Process Control Block (PCB)

The PCB is the record the OS keeps for every process. On Linux it is the structure task_struct. It holds:

PCB field	Contents
Process ID and parent ID	PID, PPID, user ID
Process state	new, ready, running, waiting or terminated
Program counter	Address of the next instruction
CPU registers	Accumulators, stack pointer, flags – saved at a context switch
CPU-scheduling information	Priority, scheduling queue pointers, time used
Memory-management information	Page table, base/limit registers

PCB field	Contents
Accounting information	CPU time used, time limits, job number
I/O status information	Open files and devices allocated to the process

1.3 Program using fork() and observation with ps and /proc

The program creates n child processes with fork(). Each child sleeps for 3 seconds, and the parent runs ps and reads /proc/<pid>/status of the first child.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main(int argc, char **argv)
{
    int n = argc > 1 ? atoi(argv[1]) : 3;
    pid_t kids[2100];
    printf("parent pid = %d\n", getpid());
    for (int i = 0; i < n; i++) {
        pid_t pid = fork();
        if (pid == 0) { sleep(3); _exit(0); }
        kids[i] = pid;
        printf("created child %d with pid %d\n", i + 1, pid);
    }
    fflush(stdout);
    if (n <= 5) {
        char cmd[100];
        snprintf(cmd, sizeof cmd, "ps -o pid,ppid,stat,vsz,rss,cmd --ppid %d",
getpid());
        sleep(1);
        system(cmd);
        snprintf(cmd, sizeof cmd, "grep -E 'Name|State|Pid|PPid|VmRSS|voluntary'
/proc/%d/status", kids[0]);
        system(cmd);
    }
    for (int i = 0; i < n; i++) waitpid(kids[i], NULL, 0);
    printf("all %d children finished\n", n);
    return 0;
}
```

Compile and run: **gcc -std=c11 -Wall -Wextra q1.c -o q1 && ./q1 3**. Output obtained:

```
parent pid = 121
created child 1 with pid 122
created child 2 with pid 123
created child 3 with pid 124
  PID  PPID STAT    VSZ   RSS CMD
  122   121 S      2688   280 ./q1 3
  123   121 S      2688   280 ./q1 3
  124   121 S      2688   280 ./q1 3
Name:   q1
State:  S (sleeping)
Pid:    122
PPid:   121
VmRSS:       280 kB
voluntary_ctxt_switches:   1
nonvoluntary_ctxt_switches: 0
all 3 children finished
```

The ps output shows three children with the same parent (PPID 121) in state S (sleeping, i.e. waiting). /proc/122/status confirms the state, the parent, the memory in use (280 kB) and the number of context switches.

1.4 Evaluation – where the time goes with 2,000 processes

I measured creating 2,000 processes and 2,000 threads (each sleeping 0.3 s) on the test machine: **2,000 processes took 0.171 s, 2,000 threads took 0.051 s**.

- **Context-switch cost:** every switch saves and loads the registers and PCB and changes the page tables; it also flushes the TLB and cools the CPU cache. In my test one process-to-process switch took about 1.4 µs (2.8 µs per round trip of two switches). With 2,000 runnable processes and a time slice of a few milliseconds, the CPU spends more and more time on switching and cache misses instead of on student work.
- **Memory per process:** each process has its own page tables, kernel stack and PCB. Even a tiny process used 280 kB resident and 2.7 MB virtual; a real exam process (runtime, libraries, session data) uses many MB. 2,000 processes × 10 MB = 20 GB, which can force the server to swap and make sessions "hang".
- **I/O wait:** most sessions wait for the student or the disk, so most processes are in the Waiting state. That is harmless for the CPU, but when many of them wake up together (for example when auto-save runs at the same time) a large ready queue builds up and response time grows.

Judgement: one process per student is easy to build and gives strong isolation, but it is not a sound design for 2,000 sessions because of memory use and switching overhead.

1.5 Design decision

Criterion	One process per student	One thread per student	Fixed pool of worker processes
Memory	Highest	Lowest (shared address space)	Low and bounded
Isolation / crash	Best – one crash affects one student	Poor – a crash can kill all sessions	Medium – a crash affects only the sessions of one worker
Switch cost	Highest	Lowest	Low
Overload behaviour	Server collapses when 2,000 start together	Many threads still compete	Requests wait in a queue; load is controlled

Decision: use a **fixed pool of worker processes**, each running many lightweight threads or an event loop for its share of the sessions (for example 8 workers for 8 cores). The memory is bounded, a crash only affects one worker, and a login storm is absorbed by a queue instead of creating 2,000 new processes. Auto-save and proctoring run as separate small services so that they cannot block the exam sessions.

1.6 Alternative concept – each session as a thread

Criterion	Separate process	Thread inside one server process
Context-switch time	Switch changes the address space (page table, TLB flush): O(1) but costlier per switch	Same address space, only registers and stack change: cheaper. Measured here: processes 2.76 and 3.18 µs, threads 2.89 and 2.54 µs per round trip, so the difference was small on this machine

Criterion	Separate process	Thread inside one server process
Memory per session	Full address space and page tables: MB	A stack (a few KB to 8 MB virtual) and a small thread descriptor: KB. 2,000 threads were created in 0.051 s against 0.171 s for processes
Effect of a crash	Only that session dies	A bad pointer or an unhandled signal in one thread kills the whole server and all 2,000 sessions

Threads give better speed and use less memory but poor fault isolation; a pool of workers is the compromise.

Q2 – CPU Scheduling

> **Q2 (K5 – Evaluate)** **[8 Marks]**
>
> **Context:** Cloud video-encoding service – short jobs (thumbnails, 2 s) and long jobs (films, 20 min) share servers; small jobs wait behind large ones.
>
> **Challenge:** Compute Gantt chart, waiting, turnaround and response time under FCFS, SJF, SRTF, priority and Round Robin; implement Round Robin.
>
> **Evaluation:** effect of time quantum (1, 3, 5, 20), starvation in SJF/priority and ageing; multilevel feedback queue.
>
> **Design Decision:** recommend a policy with calculated averages.
>
> **Complete the Code:** TODO 1; run with quantum 1 and 20.
>
> **Alternative Concept:** multilevel feedback queue instead of a single Round Robin queue.

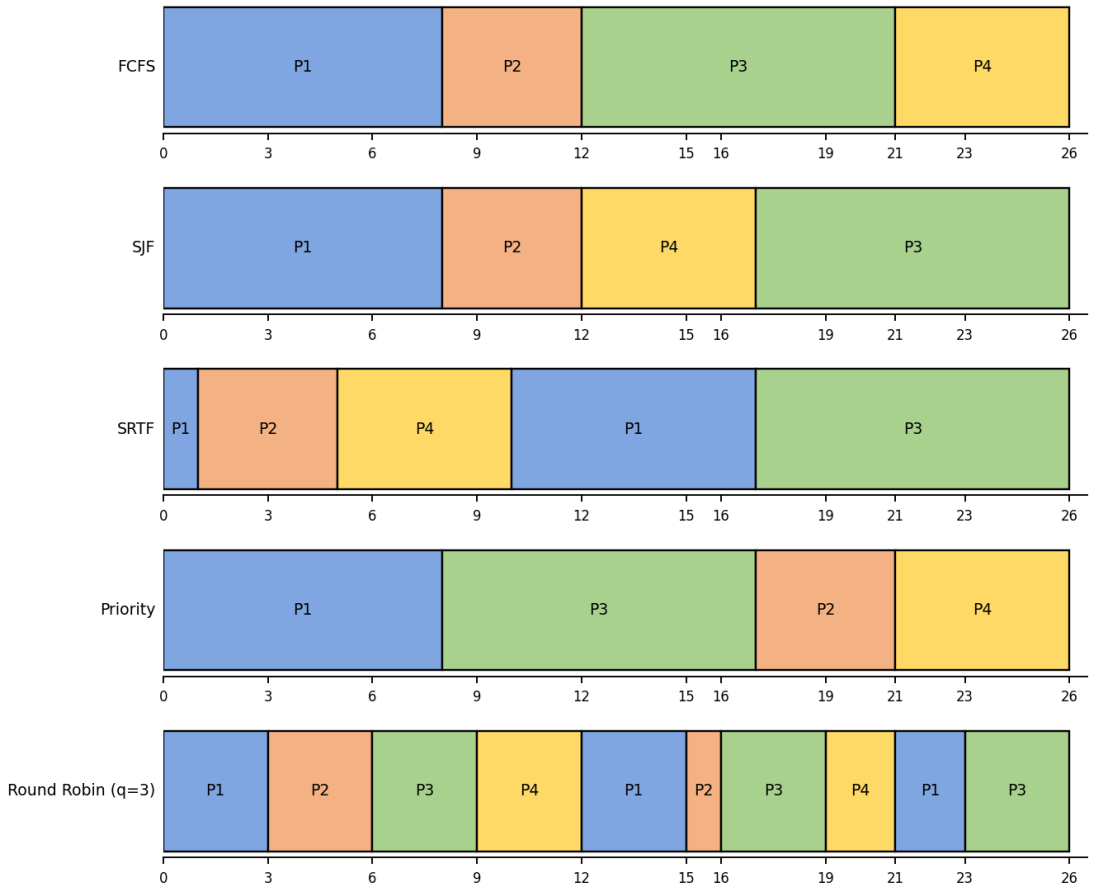
Answer:

2.1 Job set used

Process	Arrival time	Burst time	Priority (1 = highest)
P1	0	8	2
P2	1	4	3
P3	2	9	1
P4	3	5	4

The arrival and burst times are the ones in the given program; the priorities were chosen by me, because the question does not give them. Turnaround time (TAT) = finish – arrival. Waiting time (WT) = TAT – burst. Response time (RT) = first time on the CPU – arrival. SJF and priority are used in the non-pre-emptive form; SRTF is the pre-emptive form of SJF; Round Robin uses a quantum of 3.

2.2 Gantt charts



2.3 Results

Algorithm	P1 (TAT/WT/RT)	P2 (TAT/WT/RT)	P3 (TAT/WT/RT)	P4 (TAT/WT/RT)	Avg TAT / WT	Avg RT
FCFS	8 / 0 / 0	11 / 7 / 7	19 / 10 / 10	23 / 18 / 18	15.25 / 8.75	8.75
SJF	8 / 0 / 0	11 / 7 / 7	24 / 15 / 15	14 / 9 / 9	14.25 / 7.75	7.75
SRTF	17 / 9 / 0	4 / 0 / 0	24 / 15 / 15	7 / 2 / 2	13.00 / 6.50	4.25
Priority	8 / 0 / 0	20 / 16 / 16	15 / 6 / 6	23 / 18 / 18	16.50 / 10.00	10.00
Round Robin (q = 3)	23 / 15 / 0	15 / 11 / 2	24 / 15 / 4	18 / 13 / 6	20.00 / 13.50	3.00

Observation: SRTF gives the best average waiting time (6.50) and SJF is next; Round Robin has the worst waiting time but the best response time (3.00 – every job gets the CPU within a few time units). FCFS is simple but the short job P2 waits 7 units behind P1, which is exactly the complaint of the customers (the “convoy effect”).

2.4 Completed Round Robin program (TODO 1)

```
#include <stdio.h>

#define N 4

int main(int argc, char **argv)
{
    const char *name[N] = {"P1", "P2", "P3", "P4"};
    int arrival[N] = {0, 1, 2, 3};
    int burst[N] = {8, 4, 9, 5};
    int quantum = 3;
```

```
    if (argc > 1) { quantum = 0; for (const char *s = argv[1]; *s; s++) quantum =
quantum * 10 + (*s - '0'); }

    int left[N], finish[N], queue[64], head = 0, tail = 0, added[N] = {0};
    for (int i = 0; i < N; i++) left[i] = burst[i];
    int time = 0, done = 0, switches = 0;
    queue[tail++] = 0;
    added[0] = 1;
    printf("Gantt:");
    while (done < N) {
        int p = queue[head++];
        int run = left[p] < quantum ? left[p] : quantum;
        printf(" %s(%d-%d)", name[p], time, time + run);
        time += run;
        left[p] -= run;
        switches++;
        for (int i = 0; i < N; i++)
            if (!added[i] && arrival[i] <= time) { queue[tail++] = i; added[i] =
1; }
        if (left[p] > 0) queue[tail++] = p;
        else { finish[p] = time; done++; }
    }
    double tw = 0, tt = 0;
    printf("\nProcess TAT WT\n");
    for (int i = 0; i < N; i++) {
        int tat = finish[i] - arrival[i], wt = tat - burst[i];
        tt += tat;
        tw += wt;
        printf("%-7s %4d %3d\n", name[i], tat, wt);
    }
    printf("Average TAT = %.2f, average WT = %.2f\n", tt / N, tw / N);
    printf("Time slices (dispatches) = %d, context switches = %d\n", switches,
switches - 1);
    return 0;
}
```

Besides the TODO, I added a way to give the quantum on the command line and a counter of the dispatches, so the quantum study can be done with the same program. The logic of the TODO is: run p for min(quantum, left[p]), print the Gantt entry, advance the time, add newly arrived processes to the queue, and put p back at the end if time is left, otherwise record finish[p].

Output with quantum 3 (**./q2 3**) – same as the expected output:

```
Gantt: P1(0-3) P2(3-6) P3(6-9) P4(9-12) P1(12-15) P2(15-16) P3(16-19) P4(19-21)
P1(21-23) P3(23-26)
Process TAT WT
P1        23  15
P2        15  11
P3        24  15
P4        18  13
Average TAT = 20.00, average WT = 13.50
```

2.5 Quantum study (1, 3, 5, 20)

Quantum	Avg WT	Avg TAT	Avg RT	Time slices	Context switches
1	12.50	19.00	0.75	26	25
3	13.50	20.00	3.00	10	9
5	11.00	17.50	5.50	6	5
20	8.75	15.25	8.75	4	3

Output for quantum 1 and 20 (from the program): quantum 1 gives average TAT 19.00, average WT 12.50 with 25 context switches; quantum 20 gives average TAT 15.25, average WT 8.75 with only 3 context switches, and the Gantt chart is the same as FCFS (P1(0-8) P2(8-12) P3(12-21) P4(21-26)), because every burst is shorter than 20.

Evaluation: a very small quantum gives the best response time (0.75) but costs many context switches (25 in 26 units of work); if one switch takes even 1 unit, the CPU would waste half of its time. A very large quantum makes Round Robin behave like FCFS: few switches but a poor response time (8.75). A quantum slightly larger than the typical short job (10–100 ms in real systems) is the usual compromise, so that most short jobs finish within one quantum.

2.6 Starvation and ageing

In SJF/SRTF and priority scheduling a steady stream of short or high-priority jobs can keep a long or low-priority job in the ready queue for ever – this is **starvation** (indefinite blocking). In the cloud service, a 20-minute film could wait for ever if thumbnails keep arriving. **Ageing** solves it: the priority of a job is increased by a small amount for every period of time it waits (for example one level every 10 minutes), so that sooner or later even the oldest job becomes the highest-priority one and runs.

2.7 Design decision

Based on the averages: SRTF is the best for average waiting time (6.50) but it needs to know the burst times in advance (they can only be estimated for encoding jobs), it causes frequent pre-emptions and it starves long jobs. FCFS (8.75) is simple but the short jobs wait behind the films. Round Robin treats all jobs equally and gives a good response time (3.00) but a long average turnaround (20.00), and a 20-minute film would be sliced into thousands of pieces. **Recommendation: a multilevel feedback queue with ageing** – it approximates SJF without knowing the burst times, so thumbnails finish quickly, and films are still completed because of ageing, which keeps all processors busy.

2.8 Alternative concept – multilevel feedback queue (MLFQ)

Three queues: Q0 with quantum 2, Q1 with quantum 4 and Q2 served first-come-first-served; a new job enters Q0; a job that uses up its whole quantum is moved down one queue; a higher queue always pre-empts a lower one; a job that waits too long is moved up (ageing). Pseudocode:

```
for each new job: put it at the end of Q0
loop:
    pick the first job of the highest non-empty queue L   /* O(1) per queue */
    run it for quantum[L] (or until it ends / a job arrives in a higher queue)
    if it ended: finish
    else if it used the full quantum: move it to queue L+1
    else: keep it in queue L (it was I/O bound)
    every T seconds: move long-waiting jobs up one queue   /* ageing */
```

Result for the same job set (calculated): Gantt P1(0-2) P2(2-4) P3(4-6) P4(6-8) P1(8-12) P2(12-14) P3(14-18) P4(18-21) P1(21-23) P3(23-26); average TAT 19.50, average WT 13.00, average RT 1.50.

Criterion	Single Round Robin queue (q = 3)	Multilevel feedback queue
Average WT for short jobs (P2, P4)	11 and 13	9 and 13 – short jobs get the CPU sooner (RT 1 and 3, against 2 and 6)

Criterion	Single Round Robin queue (q = 3)	Multilevel feedback queue
Average WT for long jobs (P1, P3)	15 and 15	15 and 15 – the long jobs are delayed only a little
Cost of a scheduling decision	O(1) – take the head of one queue	O(1) per queue, i.e. O(k) for k queues (k = 3 here); SJF needs O(n) to search for the shortest job
Starvation	None	Possible for long jobs if short jobs keep arriving – removed by ageing

Overall: MLFQ gives better response time to short jobs at almost no extra cost, and is the better design for the video service.

Q3 – Threads

Q3 (K5 – Evaluate) **[6 Marks]**

Context: City traffic system reads number plates from 64 cameras; the first version processes the cameras one after another and falls behind at peak hours.

Challenge: Process the frames with POSIX threads (one thread per camera, pools of 4, 8 and 16) and determine: shared and private thread resources; user-level vs kernel-level threads and the thread models; the speed-up and why it stops increasing; what happens to shared counters updated without protection.

Alternative Concept: use several processes instead of threads – creation and switching time, memory, and how results are combined.

Answer:

3.1 Program

Each “frame” takes about 5 ms of waiting for the camera (simulated with a sleep) and about 5 ms of plate-recognition computation (a loop). The program (1) processes the 64 cameras one by one, (2) uses a pool of 4, 8 and 16 worker threads that take the next camera number from a shared counter protected by a mutex, (3) uses one thread per camera, and (4) shows two threads updating a shared counter with and without a mutex.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <unistd.h>
#include <pthread.h>
#define CAMERAS 64
static double now(void){struct timespec t;clock_gettime(CLOCK_MONOTONIC,&t);return
t.tv_sec+t.tv_nsec/1e9;}
static volatile double sink;
static void ms_sleep(int ms){struct timespec
r={0,ms*1000000L};nanosleep(&r,NULL);}
static void process_frame(int cam)
{
    ms_sleep(5);                    /* 5 ms waiting for the camera frame (I/O) */
    double x = cam;                    /* about 5 ms of plate recognition (CPU)
*/
    for (long i = 0; i < 1500000; i++) x = x * 1.0000001 + 0.5;
    sink = x;
}
static int next_cam; static pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
static void *worker(void *a){(void)a;
    for(;;){pthread_mutex_lock(&m);int c=next_cam++;pthread_mutex_unlock(&m);
        if(c>=CAMERAS)break;process_frame(c);}return NULL;}
static void *one(void *a){process_frame((int)(long)a);return NULL;}
```

```
static long counter_bad, counter_good;
static void *inc(void*a){int safe=(int)(long)a;for(int
i=0;i<1000000;i++){if(safe){pthread_mutex_lock(&m);counter_good++;pthread_mutex_un
lock(&m);}else counter_bad++;}return NULL;}
int main(void)
{
    double t=now();for(int c=0;c<CAMERAS;c++)process_frame(c);double seq=now()-t;
    printf("sequential (1 thread)        : %.3f s speed-up 1.00\n",seq);
    int pools[]={4,8,16};
    for(int k=0;k<3;k++){pthread_t th[16];next_cam=0;t=now();
        for(int i=0;i<pools[k];i++)pthread_create(&th[i],NULL,worker,NULL);
        for(int i=0;i<pools[k];i++)pthread_join(th[i],NULL);
        double e=now()-t;printf("pool of %2d threads           : %.3f s speed-up
%.2f\n",pools[k],e,seq/e);}
    {pthread_t th[CAMERAS];t=now();for(long
i=0;i<CAMERAS;i++)pthread_create(&th[i],NULL,one,(void*)i);
     for(int i=0;i<CAMERAS;i++)pthread_join(th[i],NULL);double e=now()-t;
     printf("one thread per camera (64)   : %.3f s speed-up %.2f\n",e,seq/e);}
    pthread_t
a,b;pthread_create(&a,NULL,inc,(void*)0);pthread_create(&b,NULL,inc,(void*)0);pthr
ead_join(a,NULL);pthread_join(b,NULL);

pthread_create(&a,NULL,inc,(void*)1);pthread_create(&b,NULL,inc,(void*)1);pthread_
join(a,NULL);pthread_join(b,NULL);
    printf("counter without lock: %ld (expected 2000000)\ncounter with mutex  :
%ld\n",counter_bad,counter_good);
    return 0;
}
```

Compile: **gcc -std=c11 -Wall -Wextra -pthread q3.c -o q3**. Output obtained:

```
sequential (1 thread)        : 0.776 s speed-up 1.00
pool of  4 threads            : 0.433 s speed-up 1.79
pool of  8 threads            : 0.441 s speed-up 1.76
pool of 16 threads            : 0.439 s speed-up 1.77
one thread per camera (64)   : 0.464 s speed-up 1.67
counter without lock: 1000000 (expected 2000000)
counter with mutex  : 2000000
```

3.2 Speed-up and why it stops increasing

Version	Time (s)	Speed-up	Remark
Sequential	0.776	1.00	Baseline
Pool of 4	0.433	1.79	Best
Pool of 8	0.441	1.76	No gain
Pool of 16	0.439	1.77	No gain
64 threads (1 per camera)	0.464	1.67	Slightly worse – thread creation and switching overhead

The speed-up stops at about 1.8 and does not grow with more threads. The reason (Amdahl's law): the waiting part of the work (about 50%) can be overlapped by several threads, but the computing part needs the CPU, and my test machine has only **one CPU core** (nproc = 1). The maximum speed-up is therefore about 1 / (0.5 + 0.5/∞) = 2, and the result 1.8 is close to this limit. Once the CPU is fully busy, more threads only add creation and switching overhead. On a computer with 4 or 8 cores the computing part would also be divided and the speed-up would continue to grow until the number of threads is about the number of cores (for CPU-bound work); for a different machine the test must be repeated.

3.3 Resources shared and private to threads

Shared by all threads of a process	Private to each thread
Code (text) section	Thread ID
Data section – global and static variables	Program counter and CPU registers
Heap memory	Stack (local variables, return addresses)
Open files and sockets, signal handlers	Thread-local storage and its own errno
Process ID and user ID	Scheduling priority and state

3.4 User-level and kernel-level threads; thread models

- **User-level threads** are managed by a thread library in user space; the kernel does not know about them. Creation and switching are very fast (no system call), but if one thread blocks in a system call the whole process blocks, and they cannot run on several CPUs.
- **Kernel-level threads** are created and scheduled by the OS. A blocking thread does not block the others and they can run on several cores, but creation and switching need system calls and cost more.

Model	Meaning	Advantage / disadvantage
Many-to-one	Many user threads are mapped to one kernel thread	Fast and cheap; one blocking call blocks all; no multi-core use
One-to-one	Each user thread has its own kernel thread (Linux pthreads, Windows)	Real parallelism and no blocking of others; each thread needs a kernel thread, so the number is limited
Many-to-many	Many user threads are multiplexed on a smaller or equal number of kernel threads	Flexible and parallel; complex to implement (two-level model)

3.5 Shared counter without protection

Two threads each added 1 to a shared counter 1,000,000 times. Without a lock, the first run gave **1,000,000 instead of 2,000,000** – half of the updates were lost; in the next two runs the result was correct by chance. With a mutex the result was always 2,000,000. The reason is that counter++ is three machine steps (load, add, store); if a thread is pre-empted after the load, the other thread's update is overwritten. This is a **race condition**: the result depends on the timing, so it can look correct in some runs, which makes the bug dangerous. The update must be a critical section protected by a mutex or an atomic operation.

3.6 Alternative concept – several processes instead of threads

Criterion	Processes	Threads
Creation time	2,000 processes: 0.171 s (about 85 µs each); with 1,000 fork: 0.0596 s	2,000 threads: 0.051 s (about 25 µs each)
Switching	Address-space switch and TLB flush (a little slower)	Only registers and stack
Memory	Separate address space for each; copy-on-write helps at the start, but writes make copies. With 64 cameras this means 64 copies of page tables and data	Shared code, data and heap; only a stack per thread

Criterion	Processes	Threads
Combining results	Needs IPC: pipes, message queues, shared memory (kernel copies, extra synchronisation)	Direct through shared variables (protected by a mutex)
Safety	A crash in one process does not affect the others	One bad thread can bring the whole program down

For 64 camera frames whose results must be collected in one place, threads are the better choice (cheap and easy to share the results); processes would be chosen only if isolation between cameras was the main requirement.

Q4 – Inter-Process Communication

Q4 (K5 – Evaluate) [6 Marks]

Context: Weather station – one process reads sensors, another analyses the readings; data must be exchanged continuously without loss.

Challenge: Implement with a pipe, then shared memory or a message queue; determine how a pipe signals end of data, shared memory vs message passing, which method copies the data, which needs extra synchronisation.

Growth Investigation: time to transfer 1, 10 and 100 MB.

Complete the Code: TODO 1; remove close(fd[1]) in the parent and explain.

Alternative Concept: shared memory with a semaphore instead of a pipe.

Answer:

4.1 Pipe implementation (completed code)

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

/* The parent (sensor reader) sends readings to the child (analyser) through a
pipe. */
int main(void)
{
    int fd[2];
    if (pipe(fd) == -1) { perror("pipe"); return 1; }
    pid_t pid = fork();
    if (pid == 0) {
        close(fd[1]);                          /* child only reads */
        int r, count = 0, sum = 0, max = 0, v;
        while ((r = read(fd[0], &v, sizeof v)) > 0) {
            count++;
            sum += v;
            if (v > max) max = v;
        }

        printf("analyser: %d readings, average %.1f, max %d\n", count,
               count ? (double)sum / count : 0.0, max);
        close(fd[0]);
        fflush(stdout);
        _exit(0);
    }
    close(fd[0]);                              /* parent only writes */
    int readings[] = {31, 33, 36, 34, 41, 38};
    for (int i = 0; i < 6; i++)
        if (write(fd[1], &readings[i], sizeof readings[i]) < 0) perror("write");
    close(fd[1]);                              /* signals end of data to the child
*/
```

```
    waitpid(pid, NULL, 0);
    printf("reader: done\n");
    return 0;
}
```

Compile and run: **gcc -std=c11 -Wall -Wextra q4.c -o q4 && ./q4**. Output (same as expected):

```
analyser: 6 readings, average 35.5, max 41
reader: done
```

The TODO is a loop: read() one int at a time from fd[0] until it returns 0; for each reading the count, the sum and the maximum are updated. The check: 31+33+36+34+41+38 = 213, and 213 / 6 = 35.5.

4.2 How the pipe signals end of data

read() on a pipe returns 0 (end-of-file) only when **all copies of the write end** of the pipe are closed and the pipe is empty. When the parent has written all readings it closes fd[1]; the child, which closed its own copy at the start, then gets 0 from read() and leaves the loop. If the pipe is empty but a writer still exists, read() blocks and waits.

4.3 Why the child never finishes when close(fd[1]) is removed in the parent

I removed the line **close(fd[1])** in the parent and ran the program. It printed nothing and was still running when it was stopped after 3 seconds by **timeout 3 ./q4b** (exit code 124). The parent still has the write end open, so the child's read() never returns 0; the child waits for more data that never comes. The parent is in waitpid(), waiting for the child to end – so both processes wait for each other: a **deadlock**.

4.4 Shared memory and message passing

Point	Shared memory	Message passing (pipe, message queue)
Model	The processes map the same memory region and read and write it like normal memory	The processes send and receive messages through the kernel (send/receive, write/read)
Copying	No copy for the data (after the setup, only one copy by the writer into the region)	Two copies: user buffer → kernel buffer → user buffer
Kernel involvement	Only for setting up the region; later none	For every message (system call)
Synchronisation	Needed – the processes must make sure that the reader does not read half-written data and that data are not overwritten (semaphores, mutex)	Provided automatically by the kernel: read blocks while empty and write blocks when full
Best for	Large amounts of data, fast exchange	Small messages, simple and safe programming, also across machines (sockets)

4.5 Growth investigation – transfer of 1, 10 and 100 MB

A parent sends the data to a child in blocks of 64 KB: (a) through a pipe, (b) by writing into a shared-memory region protected by two semaphores (one block at a time). Program q4m.c; results:

Data size	Pipe	Shared memory + semaphores	Shared memory faster by
1 MB	0.0004 s (2,244 MB/s)	0.0003 s (3,875 MB/s)	1.7x

Data size	Pipe	Shared memory + semaphores	Shared memory faster by
10 MB	0.0026 s (3,918 MB/s)	0.0010 s (9,895 MB/s)	2.5x
100 MB	0.0217 s (4,603 MB/s)	0.0089 s (11,267 MB/s)	2.4x

Evaluation: both times grow linearly with the data size (about 10 times more data takes about 10 times more time), i.e. O(n). Shared memory is about 2.4 times faster for the larger sizes because the pipe copies every byte twice and needs a system call for every 64 KB block in both directions, whereas shared memory makes one copy and uses the kernel only for semaphore operations. For 1 MB the difference is small, since the fixed costs dominate. These times are for one machine and depend on the CPU and the block size.

4.6 Alternative concept – shared memory with a semaphore for the readings

```
/* shared region */  struct { sem_t full, empty; int value; } *sh = mmap(...
MAP_SHARED | MAP_ANONYMOUS ...);
sem_init(&sh->empty, 1, 1);  sem_init(&sh->full, 1, 0);

/* reader process (sensor) */          /* analyser process */
for each reading v:                    for k = 1 .. n:
    sem_wait(&sh->empty);                  sem_wait(&sh->full);
    sh->value = v;                         v = sh->value;
    sem_post(&sh->full);                   sem_post(&sh->empty); analyse(v)
```

Criterion	Pipe	Shared memory + semaphore
Time to transfer n readings	O(n) system calls and 2 copies per reading (write + read) – higher constant	O(n) but no kernel copy; only the two semaphore calls per reading (these are cheap when no one has to wait)
Synchronisation	Automatic (blocking read/write)	Programmer must use the semaphores correctly; an error causes lost or repeated readings
Space	A kernel pipe buffer (64 KB in Linux)	The shared region (here one slot; a ring buffer of several slots is better)

Choice: for six readings per second the pipe is enough: it is simpler and safer, and the time is irrelevant. For high-rate sensor data (MB/s) shared memory with a ring buffer and semaphores is the better choice.

Q5 – Synchronisation: Critical Sections, Semaphores and Monitors

Q5 (K5 – Evaluate) **[11 Marks]**

Context: Railway ticket booking – thousands of agents book the same seats; without synchronisation a seat is sold twice; with one global lock the system is slow.

Principle given: a solution to the critical-section problem must guarantee mutual exclusion, progress and bounded waiting.

Challenge: Peterson's solution (why it works and why it may fail on modern processors); counting and binary semaphores for the seat counter; bounded buffer, readers–writers and dining philosophers with semaphores; how a monitor with condition variables makes it easier.

Test: bounded buffer with 1 and 4 producers/consumers – each item exactly once.

Deployment Decision: one lock for the system, per train or per coach – measure throughput and defend.

Complete the Code: TODO 1 and TODO 2; run 100 times; remove the mutex.

Alternative Concept: atomic operations (atomic_fetch_add) or a spin lock – time per update with 1, 4 and 16 threads.

Answer:

5.1 The three requirements

- **Mutual exclusion:** if one process is in its critical section (for example changing the seat table), no other process may be in its critical section.
- **Progress:** if no process is in the critical section and some want to enter, only those that want to enter decide who goes in, and the decision cannot be postponed for ever.
- **Bounded waiting:** after a process has asked to enter, there is a limit on how many times others can enter before it – so no agent waits for ever.

5.2 Peterson's solution

```
int flag[2] = {0, 0}; int turn;
/* process i (j = 1 - i) */
flag[i] = 1;      /* I want to enter */
turn = j;         /* but let the other go first */
while (flag[j] && turn == j) ;   /* wait */
    /* critical section */
flag[i] = 0;
```

- **Mutual exclusion:** both can be in the critical section only if flag[0] = flag[1] = 1 and the loop test is false for both; that would need turn = 0 and turn = 1 at the same time, which is impossible because turn has only one value (the one that wrote turn last waits).
- **Progress:** if the other process is not interested (flag[j] = 0), the loop test is false and i enters at once; if both are interested, turn is fixed by the last writer, so one of them enters.
- **Bounded waiting:** after leaving, process j sets flag[j] = 0, or if it tries again it sets turn = i first, so i can enter after at most one entry of j.

Why it may fail on modern processors: the proof assumes that the instructions run in the written order and that a write is seen by the other CPU immediately. Modern processors and compilers reorder instructions (for example a load of flag[j] can be done before the store of turn, and stores wait in a store buffer), and the caches of different cores may show an old value. Then both processes can see the other's flag as 0 and enter together. The algorithm only works with memory barriers or atomic instructions, which is why real systems use hardware instructions (test-and-set, compare-and-swap) and OS locks.

5.3 Semaphores and the seat counter

A semaphore S is an integer accessed only through two atomic operations: **wait(S)**: while S ≤ 0 wait; S = S – 1. **signal(S)**: S = S + 1 (and wake a waiting process).

- **Binary semaphore (mutex)**, initial value 1: protects the critical section: wait(mutex); if (seats > 0) seats = seats – 1; signal(mutex). Only one agent at a time can read and change the counter, so the same seat is never sold twice.
- **Counting semaphore**, initial value = number of free seats: an agent does wait(seats_free); if it succeeds, a seat is reserved; when a ticket is cancelled, signal(seats_free). The semaphore itself counts the seats, and an agent blocks (or tries with a non-blocking wait) when the train is full.

5.4 Classic problems with semaphores

(a) Bounded buffer (producer–consumer) – semaphores: empty = N, full = 0, mutex = 1.

```
producer:  wait(empty); wait(mutex); put item; signal(mutex); signal(full);
consumer:  wait(full);  wait(mutex); take item; signal(mutex); signal(empty);
```

The producer blocks when the buffer is full and the consumer blocks when it is empty; the mutex protects the buffer indexes.

(b) Readers–writers (readers have priority) – semaphores: rw_mutex = 1, mutex = 1; integer read_count = 0.

```
writer:  wait(rw_mutex); write; signal(rw_mutex);
reader:  wait(mutex); read_count++; if (read_count == 1) wait(rw_mutex);
signal(mutex);
         read;
         wait(mutex); read_count--; if (read_count == 0) signal(rw_mutex);
signal(mutex);
```

Many readers (for example agents checking seat availability) can read together, and a writer (the booking) has exclusive access. The first reader locks out writers and the last reader lets them in; writers may starve.

(c) Dining philosophers – five chopsticks, semaphore chopstick[5], each = 1.

```
philosopher i:  wait(chopstick[i]); wait(chopstick[(i+1) % 5]);
                eat;
                signal(chopstick[i]); signal(chopstick[(i+1) % 5]);
```

This guarantees that two neighbours do not eat together, but if all five take their left chopstick at the same time, it deadlocks. Remedies: allow at most four philosophers at the table (another semaphore initialised to 4), pick up both chopsticks only if both are free (inside a critical section), or make one philosopher take the chopsticks in the opposite order (resource ordering).

5.5 Monitor with condition variables

A monitor is a language/library construct that contains the shared data and the procedures that work on it, and only **one process can be active inside the monitor at a time** – the mutual exclusion is automatic. Condition variables (wait and signal) let a process wait inside the monitor until a condition is true. Bounded buffer as a monitor:

```
monitor BoundedBuffer {
    int count = 0; condition not_full, not_empty;
    void put(item x)  { if (count == N) not_full.wait();  insert(x); count++;
not_empty.signal(); }
    item get()        { if (count == 0) not_empty.wait(); x = remove(); count--;
not_full.signal(); return x; }
}
```

It is easier to get right than raw semaphores: the programmer cannot forget a wait(mutex) or signal(mutex), or reverse their order (a mistake that causes deadlock or no mutual exclusion), and the conditions are stated in readable form. In C with pthreads the same is done with a mutex and pthread_cond_wait/pthread_cond_signal (the wait must be in a while loop).

5.6 Completed bounded-buffer program (TODO 1 and TODO 2)

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define SIZE 5
#define ITEMS 20
```

```
int buffer[SIZE], in = 0, out = 0;
sem_t empty, full;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
long consumed_sum = 0;

void *producer(void *arg)
{
    (void)arg;
    for (int item = 1; item <= ITEMS; item++) {
        sem_wait(&empty);
        pthread_mutex_lock(&lock);
        buffer[in] = item;
        in = (in + 1) % SIZE;
        pthread_mutex_unlock(&lock);
        sem_post(&full);
    }
    return NULL;
}

void *consumer(void *arg)
{
    (void)arg;
    for (int k = 0; k < ITEMS; k++) {
        sem_wait(&full);
        pthread_mutex_lock(&lock);
        int item = buffer[out];
        out = (out + 1) % SIZE;
        pthread_mutex_unlock(&lock);
        sem_post(&empty);
        consumed_sum += item;
    }
    return NULL;
}

int main(void)
{
    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
    pthread_t p, c;
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    printf("items consumed: %d, sum = %ld (expected %d)\n", ITEMS, consumed_sum,
           ITEMS * (ITEMS + 1) / 2);
    sem_destroy(&empty);
    sem_destroy(&full);
    return 0;
}
```

Compile and run: **gcc -std=c11 -Wall -Wextra -pthread q5.c -o q5 && ./q5**. Output (same as expected):

```
items consumed: 20, sum = 210 (expected 210)
```

100 runs (loop: for i in $(seq 100); do ./q5; done | sort | uniq -c) gave: **100 items consumed: 20, sum = 210 (expected 210)** – the sum was 210 in all 100 runs.

5.7 Test with 1 and 4 producers/consumers; effect of removing the mutex

The extended program q5m.c makes each producer create its own item numbers and each consumer record every item it takes (20,000 items per producer). Results:

Test	Result
1 producer / 1 consumer, with mutex	20,000 produced – consumed exactly once 20,000, lost 0, duplicated 0

Test	Result
4 producers / 4 consumers, with mutex	80,000 produced – consumed exactly once 80,000, lost 0, duplicated 0
4 producers / 4 consumers, without mutex	80,000 produced – exactly once 76,158, lost 1,921, duplicated 1,921

With one producer and one consumer the semaphores alone are enough, because the producer is the only one that changes in and the consumer is the only one that changes out. With several producers or consumers, two of them can pass the semaphore at the same time and use the same buffer position: one item is overwritten (lost) and another is read twice (duplicated). So the mutex is required for several producers and consumers. (Also, in the given code consumed_sum += item is outside the lock; it is safe only because there is a single consumer – with several consumers it must be protected too.)

5.8 Deployment decision – one lock for the system, per train or per coach

Program q5l.c simulates booking agents: 20 trains × 10 coaches × 50 seats; each agent makes 200,000 random booking requests; every booking does some work (price check) inside the critical section. Throughput in million requests per second:

Locking	1 agent	4 agents	16 agents
One lock for the whole system	33.9	37.2	42.3
One lock per train	25.2	37.8	33.9
One lock per coach	26.9	42.3	45.4

In all runs the seats sold equalled the seats occupied (10,000), so **no seat was sold twice** with any of the three methods. **Important limitation:** my test machine has one CPU core, so only one thread can run at a time and the locks are hardly ever contended; therefore the three methods show almost the same throughput here (the small differences are noise). The advantage of fine-grained locks appears only on a multi-core machine: with a global lock all cores wait for one lock, so throughput stays flat as agents are added, while with per-train or per-coach locks agents working on different trains/coaches run in parallel, and throughput rises with the number of cores (this should be re-measured on a multi-core computer).

Decision: use **one lock per train** (or per coach for very busy trains). A global lock is simple but becomes the bottleneck as thousands of agents arrive. A lock per coach gives the highest parallelism but a booking of seats in several coaches needs several locks, which needs a fixed lock order to avoid deadlock and uses more memory; a lock per train is a good balance of parallelism, simplicity and no deadlock risk for single-train bookings.

5.9 Alternative concept – atomic operations or a spin lock

Instead of semaphore and mutex, the shared counter is increased with the atomic instruction atomic_fetch_add(&counter, 1) (no lock, one hardware instruction) or protected by a spin lock (atomic_flag_test_and_set in a loop, i.e. busy waiting). Measured for 2,000,000 updates per thread (the final counts were correct in all cases):

Threads	Atomic (ns/update)	Spin lock (ns/update)	Mutex (ns/update)
1	6.6	17.2	23.7

Threads	Atomic (ns/update)	Spin lock (ns/update)	Mutex (ns/update)
4	8.2	33.6	26.9
16	8.1	113.0	21.3

- **Time:** the atomic operation is the fastest (6–8 ns, nearly independent of the number of threads). The spin lock is quick when there is no competition but becomes much slower with 16 threads (113 ns per update), because threads that do not hold the lock keep running and spinning – and on one core a spinning thread even wastes its whole time slice while the lock holder is not running. The mutex stays at about 21–27 ns because waiting threads sleep.
- **CPU time wasted by spinning:** with the spin lock the waiting threads use 100% of a CPU for nothing; with a mutex they are blocked and the CPU is free for other work.
- **Long critical sections:** a long critical section (for example a seat booking with database access) must use a mutex or a semaphore, because spinning would waste a lot of CPU. Atomic operations and spin locks suit only very short operations (a counter, a pointer swap) that take a few instructions; atomics work only for a single variable.

For the seat counter alone, an atomic decrement (with a check that it does not go below zero) would be the fastest; for the booking of a seat with several steps, a mutex per train is the right tool.

Q6 – Deadlock Handling and the Banker's Algorithm (Day-4 Design Challenge)

> **Q6 (K5 – Evaluate)** **[13 Marks]**
>
> **Context:** Research centre – jobs need several shared resources at once (GPUs, software licences, high-speed storage); jobs sometimes wait for each other for ever and the cluster stops.
>
> A. Characterise deadlock (2) · B. Compare the strategies (3) · C. Apply the Banker's algorithm (4) · D. Predict before implementing (1) · E. Measure (1) · F. Defend the final design (2)
>
> **Complete the Code:** TODO 1; show by hand why P0's request (0, 2, 0) leads to an unsafe state.
>
> **Alternative Concept:** deadlock detection and recovery instead of the Banker's algorithm.

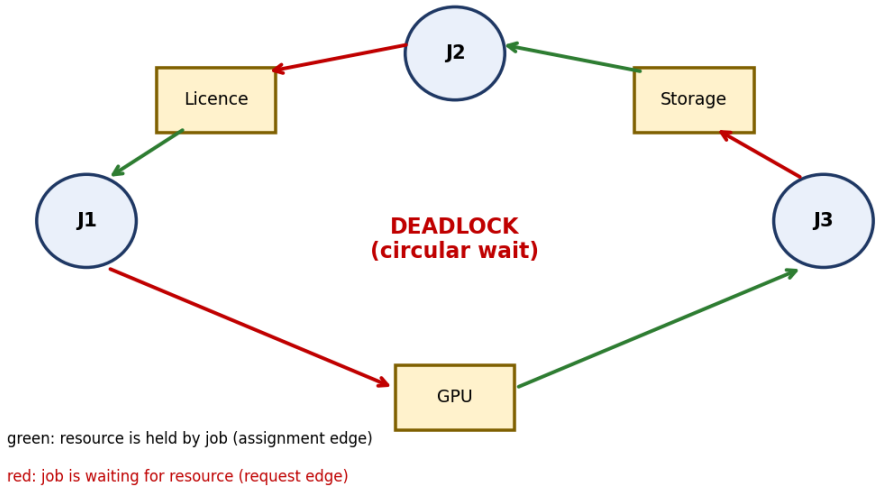
Answer:

A. Characterise Deadlock (2 Marks)

A deadlock is a situation in which a set of jobs is waiting for resources held by each other, so none of them can ever continue. It can occur only if all four conditions hold at the same time:

Necessary condition	Meaning	In the research cluster
1. Mutual exclusion	A resource can be used by only one job at a time	A GPU or a licence seat is used by one job at a time
2. Hold and wait	A job holds some resources while waiting for others	A job holds a GPU and waits for a software licence
3. No pre-emption	A resource cannot be taken away from a job; it is released only by the job	A running simulation cannot be forced to give up its GPU without losing work
4. Circular wait	A cycle of jobs exists in which each waits for a resource held by the next	J1 → GPU → J3 → Storage → J2 → Licence → J1

Resource-allocation graph: circles are jobs and rectangles are resources (one unit each). A green arrow from a resource to a job means "the job holds the resource" (assignment edge); a red arrow from a job to a resource means "the job is waiting for the resource" (request edge).



J1 holds the Licence and requests the GPU; J3 holds the GPU and requests the Storage; J2 holds the Storage and requests the Licence. The graph contains a cycle, and since every resource has only one unit, the cycle means a deadlock: all four conditions are present (the resources are not shareable, jobs hold and wait, nothing is pre-empted, and the wait is circular). If any one condition is removed, the deadlock cannot occur.

B. Compare the Strategies (3 Marks)

Strategy	Idea	Advantages for the cluster	Disadvantages
Prevention	Make one of the four conditions impossible: e.g. a job requests all resources at once (no hold and wait); numbering of resources and requesting in order (no circular wait); allow pre-emption of resources	No deadlock is possible; simple to check; resource ordering costs almost nothing	Low resource use (a job holds a GPU while waiting for the licence); starvation; jobs must know all they need in advance; pre-emption of GPUs is costly
Avoidance (Banker's algorithm)	Grant a request only if the system stays in a safe state	Better utilisation than prevention; no deadlock; no jobs are killed	Needs the maximum demand of every job in advance; the number of jobs and resources must be fixed; cost $O(m \times n^2)$ for each request; conservative – a job may wait although no deadlock would occur
Detection and recovery	Allow deadlocks; run a detection algorithm periodically; recover by aborting jobs or pre-empting resources	Highest utilisation; no advance knowledge needed; no cost for each request	Work is lost when a job is aborted; the cluster is stopped until the next detection; recovery is complex
Ignore (ostrich)	Assume that deadlock is rare and restart manually	No overhead; used by UNIX/Windows for ordinary programs	The research cluster already stops from time to time and needs an administrator – not acceptable

Evaluation for the cluster: the jobs are long and expensive, so aborting a three-day simulation (detection and recovery) loses a lot of work. The ostrich approach is already failing. Prevention wastes expensive GPUs. Avoidance is attractive if the jobs declare their maximum needs when they are submitted, which is normal on a batch scheduler (the user specifies the number of GPUs and licences in the job script).

C. Apply the Banker's Algorithm (4 Marks)

Safety algorithm: work = available; finish[] = false; look for a job i with finish[i] = false and need[i] ≤ work (need = max – allocation); if found, work = work + allocation[i], finish[i] = true and repeat; if no such job exists and some finish[i] is false, the state is unsafe; if all are true, the state is safe and the order of selection is a safe sequence. **Resource-request algorithm:** for a request of job i: (1) if request > need[i], error (more than the declared maximum); (2) if request > available, the job must wait; (3) pretend to grant: available –= request, allocation[i] += request, need[i] –= request; run the safety algorithm; if safe, the request is granted; otherwise the old state is restored and the job waits (denied).

Completed code (TODO 1 only – the rest is as given):

```
#include <stdio.h>
#include <string.h>

#define P 5
#define R 3

int avail[R] = {3, 3, 2};
int maxm[P][R] = {{7, 5, 3}, {3, 2, 2}, {9, 0, 2}, {2, 2, 2}, {4, 3, 3}};
int alloc[P][R] = {{0, 1, 0}, {2, 0, 0}, {3, 0, 2}, {2, 1, 1}, {0, 0, 2}};

/* Returns 1 if the state is safe and stores a safe sequence in seq. */
int is_safe(int seq[P])
{
    int work[R], finish[P] = {0}, count = 0;
    memcpy(work, avail, sizeof work);
    while (count < P) {
        int found = 0;
        for (int p = 0; p < P; p++) {
            if (finish[p]) continue;
            int ok = 1;
            for (int r = 0; r < R; r++)
                if (maxm[p][r] - alloc[p][r] > work[r]) { ok = 0; break; }
            if (ok) {
                for (int r = 0; r < R; r++) work[r] += alloc[p][r];
                finish[p] = 1;
                seq[count++] = p;
                found = 1;
            }
        }
        if (!found) return 0;
    }
    return 1;
}

void request(int p, const int req[R])
{
    for (int r = 0; r < R; r++)
        if (req[r] > maxm[p][r] - alloc[p][r] || req[r] > avail[r]) {
             printf("P%d request: must wait (not available)\n", p);
             return;
        }
```

```
    for (int r = 0; r < R; r++) { avail[r] -= req[r]; alloc[p][r] += req[r]; }
    int seq[P];
    if (is_safe(seq)) {
        printf("P%d request: granted, safe sequence", p);
        for (int i = 0; i < P; i++) printf(" P%d", seq[i]);
        printf("\n");
    } else {
        for (int r = 0; r < R; r++) { avail[r] += req[r]; alloc[p][r] -= req[r]; }
        printf("P%d request: denied (unsafe)\n", p);
    }
}

int main(void)
{
    int seq[P];
    if (is_safe(seq)) {
        printf("initial state safe:");
        for (int i = 0; i < P; i++) printf(" P%d", seq[i]);
        printf("\n");
    }
    request(1, (int[R]){1, 0, 2});
    request(4, (int[R]){3, 3, 0});
    request(0, (int[R]){0, 2, 0});
    return 0;
}
```

Compile and run: **gcc -std=c11 -Wall -Wextra q6.c -o q6 && ./q6**. Output (same as expected):

```
initial state safe: P1 P3 P4 P0 P2
P1 request: granted, safe sequence P1 P3 P4 P0 P2
P4 request: must wait (not available)
P0 request: denied (unsafe)
```

Hand check of the safe sequence of the initial state. Need = Max – Allocation: P0 (7,4,3), P1 (1,2,2), P2 (6,0,0), P3 (0,1,1), P4 (4,3,1); available = (3,3,2).

Step	Job chosen	Condition need ≤ work	Work after the job finishes
1	P1	(1,2,2) ≤ (3,3,2)	(3,3,2) + (2,0,0) = (5,3,2)
2	P3	(0,1,1) ≤ (5,3,2)	(5,3,2) + (2,1,1) = (7,4,3)
3	P4	(4,3,1) ≤ (7,4,3)	(7,4,3) + (0,0,2) = (7,4,5)
4	P0	(7,4,3) ≤ (7,4,5)	(7,4,5) + (0,1,0) = (7,5,5)
5	P2	(6,0,0) ≤ (7,5,5)	(7,5,5) + (3,0,2) = (10,5,7)

The safe sequence is **P1, P3, P4, P0, P2** – the same as the program. The total resources are (10,5,7), which equals the final work vector, so the check is consistent.

Decisions on the three requests (in order, each on the state left by the previous one):

- **P1 requests (1,0,2):** (1,0,2) ≤ need (1,2,2) and ≤ available (3,3,2). After pretending to grant: available (2,3,0), P1 allocation (3,0,2), need (0,2,0). The safety algorithm finds P1, P3, P4, P0, P2 → **granted**.
- **P4 requests (3,3,0):** (3,3,0) ≤ need (4,3,1), but 3 > available (2,3,0) for the first resource → P4 **must wait**; no pretend-grant is needed.
- **P0 requests (0,2,0):** (0,2,0) ≤ need (7,4,3) and ≤ available (2,3,0). Pretend to grant: available = (2,3,0) – (0,2,0) = (2,1,0); P0 allocation (0,3,0), P0 need (7,2,3). The needs are now: P0 (7,2,3), P1 (0,2,0), P2 (6,0,0), P3 (0,1,1), P4 (4,3,1). Check whether any job can finish with work (2,1,0): P0 needs 7 of the first resource (7 > 2) – no; P1 needs (0,2,0) and the second resource is 2 > 1 – no; P2 needs 6 > 2 – no; P3 needs (0,1,1) and

the third resource is 1 > 0 – no; P4 needs 4 > 2 – no. **No job can finish, so the state is unsafe** → the request is **denied** and the old state is restored. (If it had been granted, every job would wait for resources that only another waiting job could release: a possible deadlock.)

Alternative concept – detection and recovery

Instead of checking every request, jobs get the resources they ask for at once. Every few minutes the OS runs a deadlock-detection algorithm: it is the safety algorithm, but it uses the current **requests** instead of the maximum needs (a job whose current request can be satisfied is assumed to finish). Jobs that cannot finish are deadlocked; the system recovers by aborting one or all of them (or by pre-empting a resource and rolling a job back).

Criterion	Banker's algorithm (avoidance)	Detection and recovery
Time per request	$O(m \times n^2)$ for every request (safety check)	O(1) – no check when a request is made
Cost of detection	None	$O(m \times n^2)$ for each run of the algorithm, but run only every few minutes (e.g. once per 5 minutes), so the total cost is much smaller
Work lost	None – no job is ever aborted for deadlock	Everything the aborted job did since its last checkpoint; the cluster is deadlocked until the next detection run
Advance knowledge	Maximum needs of each job must be known	Not needed

Detection is cheaper per request but expensive in lost work when a deadlock occurs; the Banker's algorithm costs CPU time on each request but never loses work.

D. Predict Before Implementing (1 Mark)

Prediction: the safety algorithm has an outer loop that can run up to n times (one job finishes in each pass in the worst case); in each pass it scans up to n jobs; for each job it compares m resource types. Therefore the time is **$O(m \times n^2)$**. For a constant m (here m = 3) the time should grow with the **square of n**: 10 times more jobs should make the algorithm about **100 times slower**; so n = 10, 100 and 1,000 should take times in the ratio 1 : 100 : 10,000. The space needed is $O(n \times m)$ for the matrices.

E. Measure (1 Mark)

Program q6t.c runs the safety algorithm on the worst-case input (only one job can finish in each pass, so the loop runs n times), m = 3, with many repetitions, and gives the average time per check:

n (jobs)	Time per check	Ratio to previous	Predicted ratio
10	0.00014 ms	–	–
100	0.01172 ms	84 ×	about 100 ×
1,000	0.89691 ms	77 ×	about 100 ×

Comparison: each 10-fold increase in n made the check about 80 times slower, close to the predicted 100 times and clearly far more than the 10 times that a linear algorithm would show. The result is a little below 100 because the scan skips jobs that have already finished, so the

total number of comparisons is about $n^2/2$, and because small inputs fit better in the processor cache. The prediction of quadratic growth in n is therefore confirmed. Even for 1,000 jobs one check takes less than 1 ms, so the Banker's algorithm is affordable for a cluster with hundreds of jobs.

F. Defend the Final Design (2 Marks)

Recommendation: use deadlock avoidance with the Banker's algorithm as the main strategy in the job scheduler, together with resource ordering as a simple second line of defence, and detection with job checkpoints as a safety net.

- **Why avoidance:** the jobs are long and expensive, so aborting a job (recovery) loses days of work, and doing nothing already stops the cluster. Prevention would keep GPUs idle. The Banker's algorithm lets the scheduler use the resources fully while guaranteeing that every job can finish.
- **Cost:** $O(m \times n^2)$ per request; measured less than 1 ms for 1,000 jobs with $m = 3$. A scheduler makes a decision only when a job is submitted or finishes (not thousands of times per second), so this cost is negligible.
- **Need for maximum demands in advance:** this is the main limitation. In the cluster it is acceptable because every job is submitted with a declared request (GPUs, licences, storage) – the scheduler can refuse jobs that exceed the declared maximum. The risks are that users declare too much (wasting resources) or too little (the job is killed when it exceeds its declaration), and that the number of jobs and resource units must be known and fixed; failed hardware or a changing number of licences needs the algorithm to be re-run with the new totals.
- **Other limitation:** the algorithm is conservative – it may make a job wait for a safe state although a deadlock would not have occurred, which lowers the utilisation a little.
- **Safety net:** jobs write checkpoints so that if a deadlock still occurs (a mistake in the declared maximum, a faulty node) the detection algorithm run every few minutes can abort the job and restart it from the checkpoint with little work lost.